

Temporal Platform Team Take Home Project: Health Check Service

Objective

Assess operational thinking, system design skills, and ability to deliver a working service.

Project

Build a REST API service. This service manages health checks against configured endpoints, executes those checks, stores results, and serves historical data.

User Story

As a platform engineer, I want to:

- Configure health checks against my service endpoints
- Track availability over time
- Detect state changes (healthy to unhealthy and back)
- Investigate historical outages

Acceptance Criteria

The API must support these operations:

- Register a new endpoint to monitor (URL, check interval, expected status code)
- Remove a registered endpoint
- List all registered endpoints with their current state
- Get health check history for a specific endpoint with time range filtering
- Get state transitions for an endpoint (when its state changed and to what)

Technical Requirements

Application

- Write the service in Go or Python
- Expose a REST API that returns JSON
- Execute health checks at the configured interval
- Check multiple endpoints in parallel
- Handle shutdown without losing in-flight work
- Expose health and readiness endpoints for the service itself

Infrastructure

- Provide a Dockerfile
- Provide a Helm chart that deploys the full stack (service and datastore) to a local Kubernetes cluster (minikube, kind, or k3d)
- Choose an open source datastore that runs as a container
- The service must produce logs
- The service must expose metrics

Documentation

- Provide a README that explains:
 - How to build and run the project
 - API documentation with request and response examples
 - Your architecture decisions and their tradeoffs
 - Your choice of datastore and why
- Provide a design document that answers: How would you scale this service to monitor 50,000 endpoints with 30-second check intervals across 3 regions? Address:
 - Compute and scheduling
 - Data storage and retention
 - Failure modes and recovery
 - Deployment topology

Testing

- Write unit tests for core logic

- Write at least one integration test that exercises the full path: check, store, query

Bonus Points

- A runbook for this failure scenario: "The health check service stops executing checks"
- Alerting rules for key failure modes
- Rate limiting on outbound checks
- A data retention strategy

AI Usage

You may use AI tools to assist with this project. Be ready to discuss your work in a follow-up conversation. We will ask you to walk through your design decisions and implementation choices.

Time Expectation

We expect this to take 4–6 hours of focused work. A functional MVP with a strong design document is better than a polished but incomplete solution. Tell us where you cut corners and why.